

PRODUCT REQUIREMENT DOCUMENT

ParcelResolve AI

Smart Logistics & Delivery Dispatch System

Team: GitGud

Prepared for Hackathon Pitch Submission | 2026

Product Overview

1. Executive Summary

In the fast-paced logistics industry, resolving missing or damaged package claims is a notoriously slow, manual process. Customers currently face long wait times, while customer service representatives (CSRs) waste hours verifying photos, looking up tracking numbers, and applying company policies.

The Solution: An automated, agentic workflow that integrates directly into the user's existing communication channel (WhatsApp). Utilizing Google Gemini's multimodal AI capabilities, the system autonomously extracts data from unstructured customer complaints and images, cross-references internal databases, and instantly executes resolution decisions (Refund, Redeliver, or Request More Info).

2. Problem Statement

- **High Friction for Customers:** Filing a claim usually requires navigating complex web portals, downloading separate apps, or waiting on hold.
- **Operational Bottlenecks:** Human agents must manually inspect photos of damaged goods, parse tracking numbers, and query backend databases, leading to a high Cost-Per-Contact (CPC).
- **Unstructured Data Handling:** Logistics SMEs receive orders and complaints via messy, unstructured texts and photos on WhatsApp, which traditional software cannot easily process.

3. User Personas

Persona A: The End Customer (Requester)

Goal: Wants immediate resolution for a damaged or missing package without jumping through hoops.

Behavior: Takes a quick photo of the damaged box and texts customer support via WhatsApp. Expects an instant reply.

Persona B: The Logistics Administrator (Dispatcher/CSR)

Goal: Needs to monitor incoming claims, ensure AI accuracy, and intervene only when edge cases arise.

Behavior: Watches a live dashboard to see automated resolutions. Only steps in if the system flags a high-value or highly ambiguous claim.

4. Product Scope & Vision

For the scope of this hackathon, the MVP (Minimum Viable Product) will strictly focus on the end-to-end automation of the claims reporting pipeline.

IN SCOPE

- WhatsApp messaging integration via Twilio Sandbox.
- Multimodal AI extraction (Text + Image) using Google Gemini 1.5 Flash.
- A backend mock-database (Java) for real-time order lookups.
- An automated decision engine to output a resolution format in JSON.
- Fallback logic for missing data (e.g., prompting the user for a photo if one is missing).

OUT OF SCOPE

- A standalone mobile application (we leverage existing WhatsApp UI).
- Integration with real bank APIs for actual payment processing.

5. Functional Requirements

Feature / Agent	Description	Technical Dependency
Omnichannel Entry	System must receive text and media attachments from WhatsApp.	Twilio API, Webhooks, Ngrok
Agent A: Extractor	System must analyze unstructured text and images to extract tracking numbers and	Google Gemini 1.5 Flash (Multimodal)

	assess damage severity (HIGH, MEDIUM, LOW, NONE).	
Agent B: Database Lookup	System must query an internal database using the extracted tracking number to retrieve item value and delivery status.	Java Backend (HashMap/SQL)
Agent C: Decision Maker	System must evaluate extracted damage and item value against company policy to issue a final automated decision.	Java Logic / Gemini Context
Stateful Error Handling	If a customer provides a text without a photo, the system must immediately reply asking for visual proof before proceeding.	Javalin Server Logic

6. Non-Functional Requirements

- **Performance:** The entire cycle—from WhatsApp message received to AI decision returned—must execute in under **5 seconds** to provide a seamless chat experience.
- **Reliability:** The AI prompts must enforce strict JSON outputs to prevent application crashes caused by conversational hallucinations.
- **Scalability:** The architecture must be stateless at the endpoint level, allowing multiple customer claims to be processed concurrently.

7. User Flow (The Golden Path)

1. **Trigger:** Customer sends a photo of a crushed box and texts, "My box is crushed! MY12345" to the company's WhatsApp.
2. **Ingestion:** Twilio forwards the payload to the Javalin backend via an Ngrok tunnel.
3. **Validation:** The Java server verifies both text and image are present.
4. **Extraction:** Google Gemini analyzes the media, outputting structured JSON:
{"trackingNumber": "MY12345", "damage": "HIGH"}.
5. **Contextualization:** Java queries the mock database to find MY12345 is worth RM 150 and was recently delivered.
6. **Resolution:** The decision logic approves an automatic refund and sends a confirmation WhatsApp reply back to the customer instantly.

8. Success Metrics (KPIs for Pitch)

- **Time to Resolution (TTR):** Decreased from an average of 48 hours to under 5 seconds.
- **Human Touch Rate:** Reduced by 85%, freeing up CSRs to handle complex logistics routing rather than low-level claims.
- **Data Extraction Accuracy:** Target >95% accuracy in successfully reading tracking numbers from user-submitted images.